

Descrição geral:

Esta documentação refere-se às planilhas das turmas do IGA que utilizam o ModeloSheets como biblioteca central de processamento.

O ModeloSheets é a biblioteca responsável por implementar toda a lógica do Sistema de Presenças, incluindo validações, regras de negócio, processamento de imagens, controle de duplicidade de datas, atualização da Página 1, espelhamento para Correções Manuais e aplicação da regra de desistência automática.

Cada planilha de turma funciona como um cliente da biblioteca. Essas planilhas possuem:

- Configurações específicas (nomes de abas, cores e IDs de pastas no Drive);
- Funções de integração com a biblioteca;
- Publicação do Web App;
- Menu personalizado para uso operacional;
- A aba “Lancar_Presenca”, utilizada para registrar a aula do dia, selecionar os alunos presentes e definir parâmetros como data, professor e aula duplicada.

A aba “Lancar_Presenca” é o ponto operacional do cliente, onde o lançamento é iniciado. A partir dela, a biblioteca ModeloSheets processa os dados e executa todas as rotinas necessárias no restante da planilha.

Toda a regra de negócio permanece centralizada no ModeloSheets, garantindo padronização, reutilização de código, facilidade de manutenção e escalabilidade entre múltiplas turmas do IGA.

Assim, esta documentação descreve a arquitetura, funcionamento e integração entre as planilhas das turmas (clientes) e a biblioteca ModeloSheets, que constitui o núcleo lógico do sistema.

Biblioteca (Modelo Sheets)

Arquivos	AZ +
Código.gs	
WebApp.gs	
upload.html	
Bibliotecas	+
Serviços	+
Drive	

1. Codigo.gs

Responsabilidade: Núcleo lógico do sistema de presenças.

VISÃO GERAL

Este arquivo tem toda lógica operacional do sistema:

- Lançamento de presença
- Validação de duplicidade de data
- Atualização da Página 1
- Espelhamento para Correções Manuais
- Regra automática de desistência
- Salvamento de imagens no Drive
- Processamento do Web App
- Controle de concorrência com LockService

DEPENDÊNCIAS

- SpreadsheetApp
- DriveApp
- LockService
- Utilities
- Session

Estrutura obrigatória de abas:

- Página 1
- Correções Manuais
- Lancar_Presenca
- Presenca_Dados

Objeto CONFIG definido no cliente.

FUNÇÕES

executarProcessoCompleto(ss, config)

Objetivo:

Executar todas as rotinas principais após um lançamento ou correção.

Fluxo:

1. Atualiza status geral.
2. Aplica regra automática de desistência.
3. Espelha Página 1 para Correções Manuais.
4. Recarrega alunos na aba de lançamento.
5. Usa SpreadsheetApp.flush() entre etapas para garantir consistência.

dataJaExisteNoLog(dadosSheet, dataAula)

Objetivo:

Verificar se já existe registro de presença para determinada data.

Parâmetros:

- dadosSheet: aba Presenca_Dados
- dataAula: objeto Date

Retorno:

- true se já existir
- false se não existir

Observação técnica:

A data é normalizada removendo o horário antes da comparação.

salvarPresenca(ss, config, linksImagens)

Objetivo:

Registrar oficialmente uma nova presença.

Controle:

Utiliza `LockService.getScriptLock()` para impedir execução simultânea.

Validações:

- Data preenchida
- Professor preenchido
- Data válida
- Não permite duplicidade

Fluxo:

1. Normaliza a data.
2. Verifica duplicidade no log.
3. Captura lista de presentes.
4. Registra linha no `Presenca_Dados`.
5. Atualiza Página 1.
6. Espelha Correções Manuais.
7. Aplica regra de desistência.

Retorno:

Mensagem de sucesso ou erro controlado.

atualizarPagina1ComPresencas(ss, config)

Objetivo:

Criar nova coluna de presença e marcar P ou F para cada aluno.

Funcionalidade especial:

Se o checkbox D4 (Aula Duplicada) estiver marcado:

Cria duas colunas consecutivas com a mesma data.

Fluxo:

1. Captura data de B4.
2. Verifica D4.
3. Obtém último registro do log.
4. Define quantidade de colunas (1 ou 2).
5. Para cada coluna:
 - Cria header com a data.
 - Marca P ou F.
 - Aplica cor correspondente.

Observação:

Sempre cria nova coluna ao final. Não reutiliza coluna existente.

aplicarRegraDesistenciaAutomatica(ss, config)

Objetivo:

Garantir que após um aluno receber "D", todas as presenças seguintes sejam "D".

Lógica:

- Percorre cada linha.
- Ao encontrar o primeiro "D":
Todas as colunas seguintes viram "D".

Após execução:

Revalida o status geral.

espelharCorrecoesManuais(ss, config)

Objetivo:

Criar cópia exata da Página 1 na aba Correções Manuais.

Fluxo:

1. Cria aba se não existir.
2. Limpa conteúdo anterior.
3. Copia valores.
4. Copia cores.
5. Congela linha 1 e colunas iniciais.

Observação:

É uma cópia completa, não incremental.

salvarCorrecoesManuais(ss, config)

Objetivo:

Aplicar alterações feitas manualmente na aba Correções Manuais de volta para Página 1.

Fluxo:

1. Compara célula por célula.
2. Se houver alteração:
 - Atualiza valor.
 - Atualiza cor.
 - Adiciona comentário contendo:
Data, usuário e valor antigo → novo.

Após salvar:

- Aplica regra de desistência.
- Atualiza status geral.

carregarAlunosNaPresenca(ss, config)

Objetivo:

Recarregar lista de alunos ativos na aba Lancar_Presenca.

Fluxo:

1. Limpa área A13:C100.
2. Remove checkboxes antigos.
3. Copia IDs e nomes da Página 1.
4. Insere novos checkboxes.

validarStatusGeral(ss, config)

Objetivo:

Atualizar coluna de Status (Cursando / Desistente).

Regra:

Se existir "D" em qualquer presença:

Status = Desistente

Caso contrário:

Status = Cursando

Aplica na Página 1 e em Correções.

salvarImagem(base64, tipo, dataStr, config)

Objetivo:

Salvar imagem enviada pelo Web App no Google Drive.

Processo:

1. Decodifica base64.
2. Cria Blob PNG.
3. Salva na pasta configurada.
4. Retorna URL pública do arquivo.

processarEnvioWeb(ss, config, lancheBase64, selfieBase64, listaBase64)

Objetivo:

Processar envio completo via Web App.

Controle:

Utiliza LockService.

Fluxo:

1. Captura data.
2. Normaliza.
3. Verifica duplicidade.
4. Gera string formatada da data.
5. Salva imagens.
6. Chama salvarPresenca.

2. WebApp.gs

Responsabilidade: Disponibilizar a interface do sistema via Web App (HTML).

VISÃO GERAL:

Este arquivo é responsável por expor a interface pública do sistema de presenças através do Google Apps Script Web App.

Ele não contém lógica de negócio.

Sua única função é servir o arquivo HTML da interface (upload.html).

FUNÇÕES

servirInterface()

Objetivo:

Retornar a interface HTML que será exibida quando o Web App for acessado via URL publicada.

Responsabilidade dentro da arquitetura:

É o ponto de entrada da aplicação Web.

Quando o usuário acessa o link do Web App, esta função entrega a interface visual.

Fluxo:

1. Carrega o arquivo HTML chamado "upload".
2. Define o título da aba do navegador como "Registro de Presença".
3. Adiciona meta tag viewport para responsividade (compatível com celular).
4. Permite que a página seja carregada dentro de iframe (ALLOWALL).
5. Retorna o HtmlOutput renderizado.

Detalhamento técnico:

- `HtmlService.createHtmlOutputFromFile("upload")`
Carrega o arquivo upload.html do projeto.
- `setTitle("Registro de Presença")`
Define o título da aba do navegador.
- `addMetaTag('viewport', 'width=device-width, initial-scale=1')`
Garante layout responsivo em dispositivos móveis.
- `setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL)`
Permite que o Web App seja incorporado em iframes.
Importante caso o sistema seja embedado em outro site.

Observações técnicas:

- Esta função deve estar associada ao doGet(e) quando publicado como Web App.
- Não possui validações, autenticação ou controle de acesso.
- A segurança depende da configuração de implantação do Web App (Quem pode acessar).

Papel na arquitetura geral:

Usuário acessa URL do Web App

→ servirInterface()

→ upload.html é renderizado

→ upload.html chama funções do backend (processarEnvioWeb)

3. upload.html

Responsabilidade: Interface Web para envio de fotos da presença.

VISÃO GERAL

Este arquivo é a camada de apresentação do sistema quando acessado via Web App.

Ele permite que o usuário:

- Anexe três imagens obrigatórias:
 - Foto do lanche
 - Selfie
 - Foto da lista
- Visualize prévia das imagens antes do envio
- Envie os dados para o backend (Google Apps Script)

Não contém regra de negócio.

Toda validação estrutural ocorre no backend.

ESTRUTURA GERAL

1. HTML (estrutura da interface)
2. CSS (estilização)
3. JavaScript (lógica de interação e envio)

INTERFACE (HTML)

Componentes principais:

- Título: "Registro de Presença"
- Três inputs ocultos do tipo file
- Três labels estilizadas como botões
- Três áreas de preview de imagem
- Três mensagens de confirmação
- Botão principal de envio

Campos de imagem:

- id="lanche"
- id="selfie"
- id="lista"

Todos utilizam:

accept="image/*"

Permitindo apenas seleção de imagens.

BOTÃO PRINCIPAL

Botão com id="btnSalvar"

Aciona:

salvarPresenca()

ESTILIZAÇÃO (CSS)

Características principais:

- Layout centralizado
- Card com sombra e borda arredondada
- Inputs ocultos (display: none)
- Labels estilizadas como área de upload
- Preview com limite de altura
- Botão com estados:
 - Normal
 - Hover
 - Disabled

Responsividade:

Meta tag viewport garante adaptação a dispositivos móveis.

LÓGICA JAVASCRIPT

Variável global:

```
const imagens = {  
  lanche: null,  
  selfie: null,  
  lista: null  
};
```

Armazena as imagens em formato Base64.

PROCESSAMENTO DE IMAGEM

Trecho:

```
["lanche", "selfie", "lista"].forEach(tipo => { ... })
```

Fluxo:

1. Detecta alteração no input file.
2. Obtém arquivo selecionado.
3. Exibe mensagem "Processando..."

4. Usa FileReader.
5. Converte imagem para Base64 (DataURL).
6. Armazena no objeto imagens.
7. Exibe preview.
8. Mostra mensagem "Imagem anexada".

Importante:

```
reader.readAsDataURL(file);
```

Isso gera string Base64 que será enviada ao backend.

FUNÇÃO DE ENVIO

salvarPresenca()

Objetivo:

Enviar as três imagens para o backend.

Validação inicial:

Se qualquer imagem for null:

Exibe alerta e interrompe execução.

Fluxo de envio:

1. Desabilita botão.
2. Altera texto para "Enviando dados...".
3. Chama backend usando google.script.run.

Chamado:

```
.salvarPresencaComFotos(  
  imagens.lanche,  
  imagens.selfie,  
  imagens.lista  
);
```

Handlers:

```
withSuccessHandler(res => { ... })
```

Se resposta for diferente de:

"Presença e fotos registradas com sucesso!"

Mostra erro retornado.

```
withFailureHandler(err => { ... })
```

- Exibe erro.
- Reabilita botão.
- Permite nova tentativa.

INTEGRAÇÃO COM BACKEND

Este HTML depende da função:

salvarPresencaComFotos()

Definida no Apps Script.

Fluxo completo:

Usuário anexa imagens

- Imagens convertidas para Base64
- salvarPresenca()
- google.script.run
- Backend processa
- Backend salva imagens no Drive
- Backend registra presença
- Retorna mensagem

PONTOS TÉCNICOS IMPORTANTES

- As imagens são enviadas em Base64.
- Pode gerar payload grande dependendo do tamanho da imagem.
- Não há compressão ou redimensionamento.
- Não há autenticação no front-end.
- A segurança depende da configuração do Web App.

LIMITAÇÕES ATUAIS

- Não há barra de progresso real.
- Não há tratamento de timeout.
- Não há compressão de imagem.
- Não há limite explícito de tamanho de arquivo.

Cliente (Planilha Sheets)

Arquivos	AZ +
appsscript.json	
Conexao.gs	
Bibliotecas	+
ModeloSheets	
Serviços	+
Drive	

1. Conexao.gs

Responsabilidade: Camada de integração entre a planilha do cliente e a biblioteca ModeloSheets.

VISÃO GERAL

Este arquivo é o ponto de conexão entre:

- A planilha específica do cliente
- A biblioteca ModeloSheets
- O Google Drive
- O Web App

Ele contém:

- Configurações individuais do cliente
- IDs das pastas do Drive
- Mapeamento das abas
- Funções que chamam a biblioteca
- Menu personalizado da planilha
- Ponto de entrada do Web App

Não contém regra de negócio.

A lógica principal está na biblioteca ModeloSheets.

OBJETO DE CONFIGURAÇÃO

const CONFIG

Responsabilidade:

Centralizar todas as configurações específicas da planilha atual.

Estrutura:

ABAS:

Define os nomes das abas obrigatórias.

- P1 → "Pagina 1"
- CORR → "Correções Manuais"
- LANCAR → "Lancar_Presenca"
- DADOS → "Presenca_Dados"

CORES:

Define o padrão visual do sistema.

- A → Faltas abonadas
- D → Desistente
- F → Faltas
- P → Presença

PASTAS_DRIVE:

IDs das pastas onde as imagens serão salvas.

- lanche
- selfie
- lista

Observação importante:

Esse arquivo permite reutilizar a mesma biblioteca para múltiplos clientes, alterando apenas o CONFIG.

FUNÇÕES

autorizarAcessoAoDrive()

Objetivo:

Forçar solicitação de permissão do Google Drive.

Motivação:

Quando o Web App é usado por outra conta, o Drive pode exigir autorização manual.

Fluxo:

1. Acessa a pasta raiz do Drive.
2. Tenta acessar uma pasta pelo ID.
3. Se bem-sucedido → mostra alerta de sucesso.
4. Se falhar → orienta o usuário a revisar permissões.

Função puramente administrativa.

forçarSincronizacaoGeral()

Objetivo:

Executar atualização manual da Página 1 e Correções.

Fluxo:

1. Obtém planilha ativa.
2. Chama atualizarPagina1ComPresencas.
3. Chama espelharCorrecoesManuais.
4. Exibe alerta de conclusão.

Usado para correção manual ou reprocessamento.

doGet(e)

Objetivo:

Ponto de entrada do Web App.

Função obrigatória para publicação como Web App.

Fluxo:

1. Chama ModeloSheets.servirInterface().
2. Retorna o HTML renderizado.

Sem essa função, o Web App não funciona.

salvarPresencaComFotos(lanche, selfie, lista)

Objetivo:

Receber imagens enviadas pelo HTML e repassar para a biblioteca.

Parâmetros:

- lanche (Base64)
- selfie (Base64)

- lista (Base64)

Fluxo:

1. Obtém planilha ativa.
2. Chama `ModeloSheets.processarEnvioWeb()`.
3. Retorna resultado ao front-end.

Observação:

Essa função é chamada via `google.script.run` no `upload.html`.

onOpen()

Objetivo:

Criar menu personalizado na planilha.

Executa automaticamente ao abrir a planilha.

Cria menu:

"Sistema de Presenças"

Item disponível:

- Salvar Correções Manuais

Função associada:

`rodarSalvarCorrecoes()`

processamentoDiarioCompletoPresencas()

Objetivo:

Executar processamento completo manualmente.

Fluxo:

1. Obtém planilha ativa.
2. Chama `ModeloSheets.executarProcessoCompleto()`.

Pode ser usada para automação via gatilho.

rodarSalvarCorrecoes()

Objetivo:

Executar salvamento das alterações feitas na aba Correções Manuais.

Fluxo:

1. Obtém planilha ativa.
2. Chama `ModeloSheets.salvarCorrecoesManuais()`.

PAPEL NA ARQUITETURA

Planilha do Cliente

↓

Conexao.gs

↓

Biblioteca ModeloSheets

Separação de responsabilidades:

Conexao.gs:

- Configuração específica
- Integração
- Web App
- Menu
- Permissões

ModeloSheets:

- Lógica de negócio
- Processamento
- Regras
- Validações
- Persistência

VANTAGEM ARQUITETURAL

Permite:

- Reutilizar a mesma biblioteca para múltiplos clientes.
- Alterar apenas IDs e nomes de abas.
- Manter atualização centralizada da lógica.
- Reduzir duplicação de código.

PONTOS TÉCNICOS IMPORTANTES

- Se a biblioteca for removida, o sistema quebra.
- IDs de pastas devem estar corretos.
- A publicação do Web App deve ser feita pelo dono da planilha.
- Permissões do Drive dependem da conta que implantou o script.